

TP1 – A LA DECOUVERTE DU MICROCONTROLEUR STM32F103

1 Table des matières

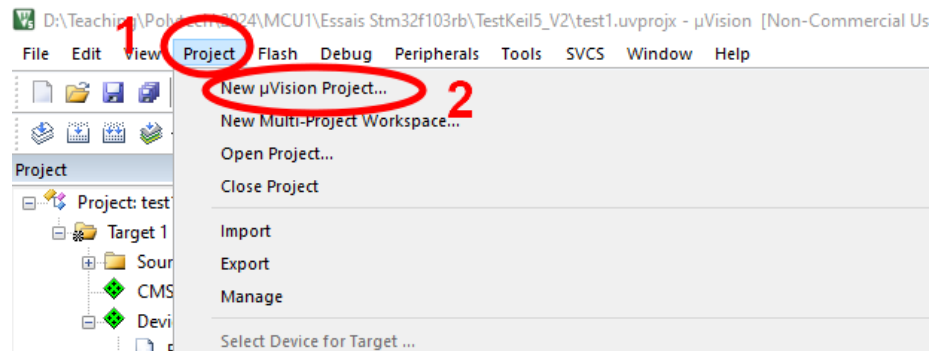
1	Création d'un projet KEIL5 pour le microcontrôleur stm32f103RB.....	2
2	Découverte de la chaîne de compilation	9
2.1	La précompilation	9
2.2	La cartographie mémoire – fichier MAP	12
3	Premiers pas avec le STM32F103RB	13
3.1	Simulation de fonctionnement du STM32F103RB.....	16
3.1.1	Simulation en mode pas à pas et vérification du contenu des registres de configuration des Pins	17

1 Création d'un projet KEIL5 pour le microcontrôleur stm32f103RB

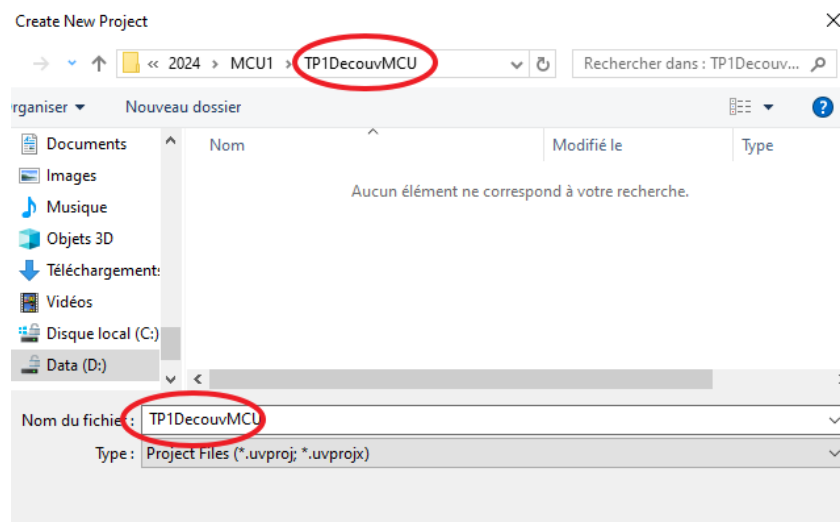
On va créer un nouveau projet :

1. **Menu Project**

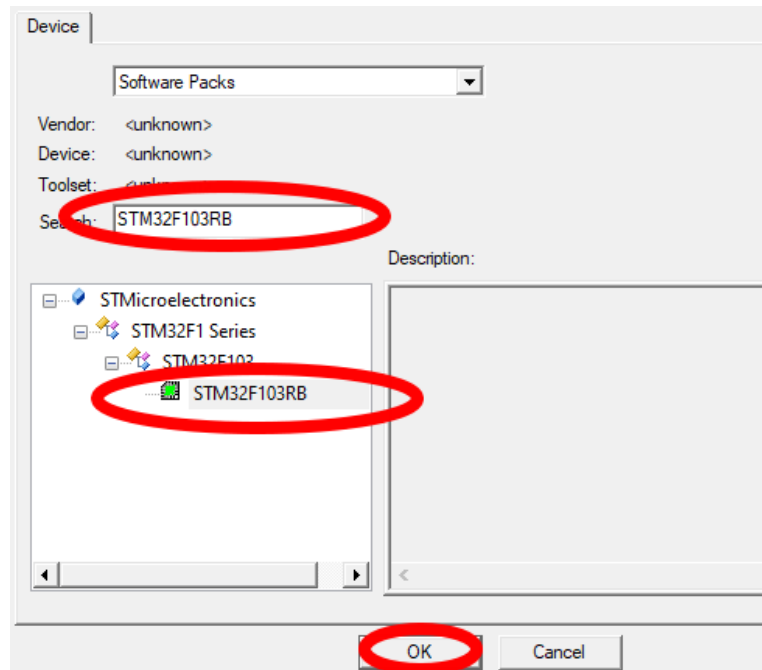
→ 2. **New µvision Project**



On pensera à créer un nouveau répertoire ou dossier où sauvegarder le **PROJET** (Astuce : utiliser le même nom que votre projet) ici **TP1DecouvMCU**



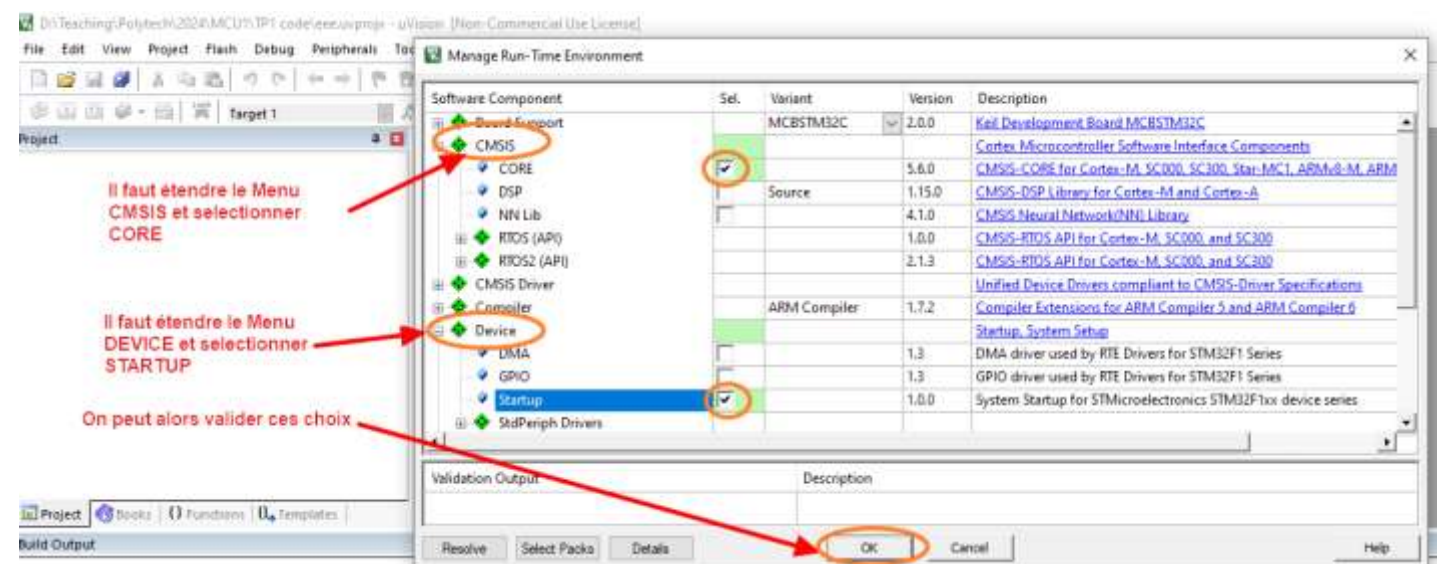
u de choix de la cible apparaît :
dans le champ **Search** taper le
nom de la cible **STM32F103RB**
Ensuite cliquer sur le processeur,
ce qui fait apparaître la
description valider ce choix de
processeur



**ARRRRGGGGGGGGG, impossible de trouver le STM32F103RB !! ??? !!! en ce cas suivre le tutoriel
AJOUTER_PACKAGES.pdf pour faire ajouter à KEIL le bon package concernant la cible choisie puis
reprendre ce tutoriel**

Pour le moment la fenêtre **Project** est vide de tout fichier.

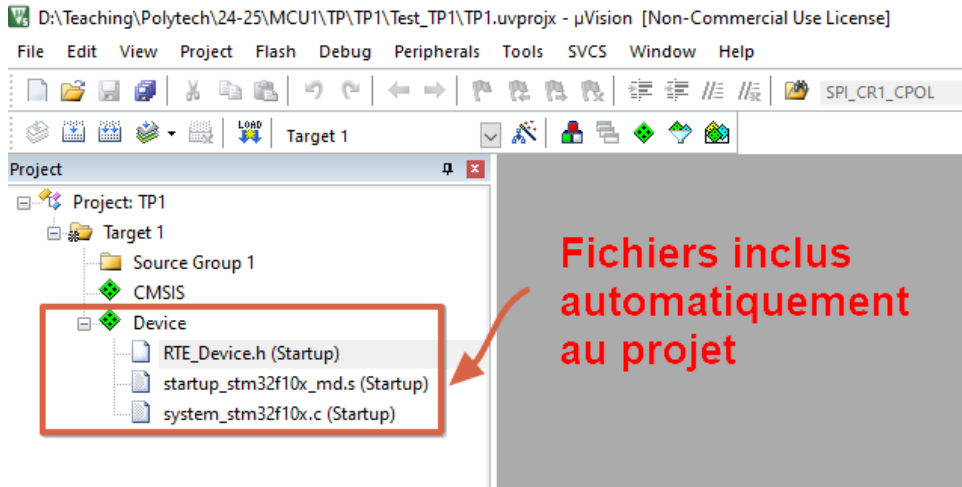
La fenêtre **Manage Run-Time Environment** apparaît, il faut Etendre les menus suivants et sélectionner les modules à inclure
dans notre nouveau projet :



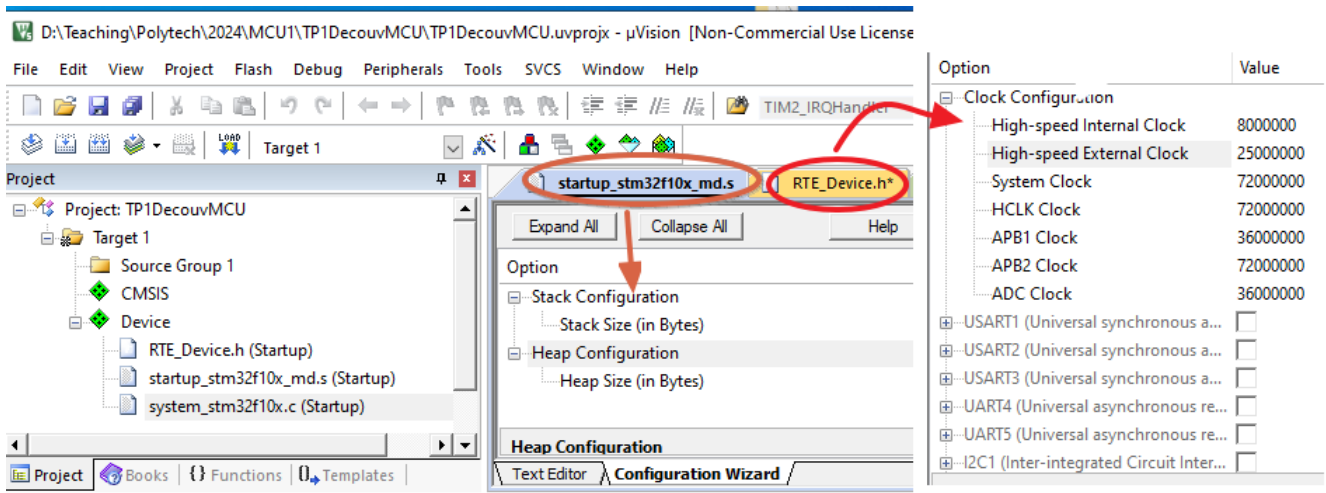
**On pourra changer à tout
moment les choix en cliquant
sur ce bouton**



le projet contient maintenant cette structure de fichiers :



Vous pouvez éditer et modifier ces fichiers avec l'éditeur de texte. Certains fichiers : **startup_stm32f10x_md.s**, **RTE_Device.h** sont modifiables via une interface graphique



Résumé sur les fichiers fournis par Keil µvision

Fichier <i>startup_stm32f103x_md.s</i>	Fichier <i>RTE_Device.h</i>	Fichier <i>system_stm32f103x.c</i>
configure les tailles de réservation mémoire de la pile et du tas	Fichier d'entête paramétrable pour permettre une modification plus aisée des valeurs de paramètres clés : - les sources et vitesses d'horloge des divers modules du processeur (cœur, USB, périphériques) - configuration de Pins et vitesses de transfert pour la liaison série	gère : - les sources et vitesses d'horloge des divers modules du processeur (cœur, USB, périphériques) - les mises sous tensions des modules - le cadencement de la mémoire flash

Si on compile maintenant, on obtient une erreur et 1 Warning



Si on compile en cliquant sur ce bouton On obtient 1 Erreur et 1 Warning

```
1 warning generated.
compiling system_stm32f10x.c...
linking...
.\Objects\TP1DecouvMCU.axf: Error: L6218E: Undefined symbol main (referred from __rtentry2.o).
Not enough information to list image symbols.
Not enough information to list load addresses in the image map.
Finished: 2 information, 0 warning and 1 error messages.
".\Objects\TP1DecouvMCU.axf" = 1 Error(s), 1 Warning(s)
Target not created.
Build Time Elapsed: 00:00:00
```

Le Warning vient de la compilation du fichier *system_stm32f10x.c*

```
Rebuild started: Project: TP1DecouvMCU
*** Using Compiler 'V6.14', folder: 'C:\Keil_v5\ARM\ARMCLANG\Bin'
Rebuild target 'Target 1'
assembling startup_stm32f10x_md.s...
RTE/Device/STM32F103RB/system_stm32f10x.c(166): warning: no previous extern declaration for non-static variable 'AHBPrescTable' [
__I uint8_t AHBPrescTable[16] = {0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 3, 4, 6, 7, 8, 9};
^
RTE/Device/STM32F103RB/system_stm32f10x.c(166): note: declare 'static' if the variable is not intended to be used outside of this
__I uint8_t AHBPrescTable[16] = {0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 3, 4, 6, 7, 8, 9};
^
1 warning generated.
compiling system_stm32f10x.c...
```

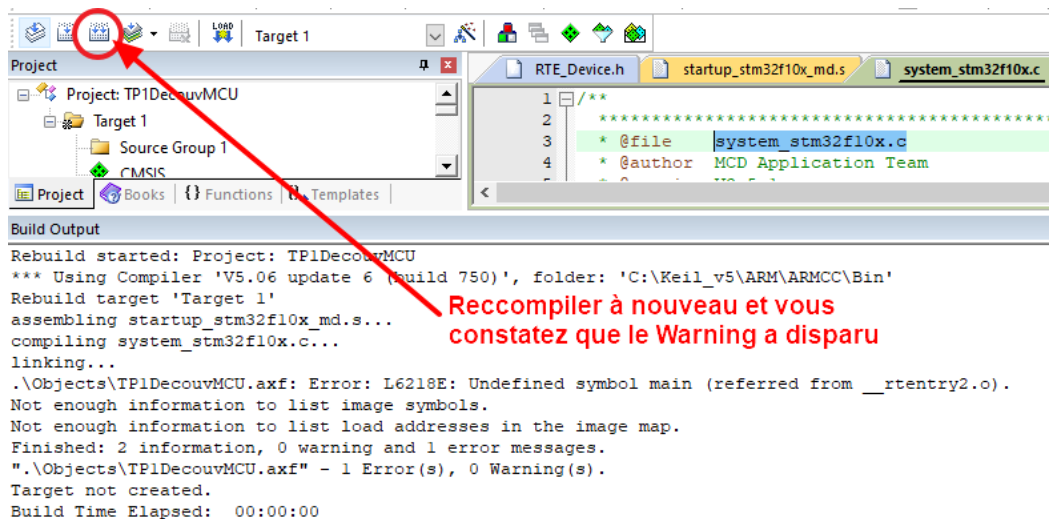
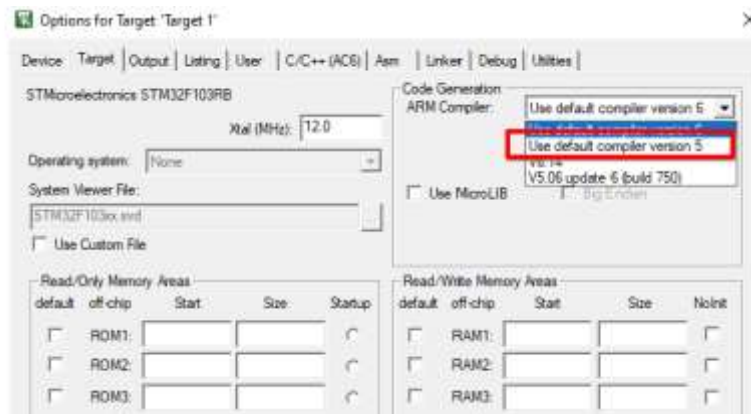
Nous allons tenter de résoudre le problème du warning en changeant de chaîne de compilation.



Pour changer de chaîne de compilation,
1 cliquez sur le bouton
"baguette magique"

2

cliquez sur l'onglet
"TARGET" puis
choisir
"use default
compiler version 5"



Les warning sont dus au choix de la chaîne de compilation. Ce qui est totalement délirant. ... c'est qu'après avoir compilé en version 5, vous pouvez revenir en version 6 et vous n'aurez plus de warning ... En fait passer en version 5 change des options qui restent quand on retourne en version 6

Nous allons maintenant résoudre le problème de l'erreur de compilation en créant une fonction `int main(void)`.

```
Rebuild started: Project: TP1DecouvMCU
*** Using Compiler 'V5.06 update 6 (build 750)', folder: 'C:\Keil_v5\ARM\ARMCC\Bin'
Rebuild target 'Target 1'
assembling startup_stm32f10x_md.s...
compiling system_stm32f10x.c...
linking...
.\Objects\TP1DecouvMCU.axf: Error: L6218E: Undefined symbol main (referred from __rtentry2.o)
Not enough information to list image symbols.
Not enough information to list load addresses in the image map.
Finished: 2 information, 0 warning and 1 error messages.
".\Objects\TP1DecouvMCU.axf" - 1 Error(s), 0 Warning(s).
Target not created.
Build Time Elapsed: 00:00:00
```

L'erreur est logique à ce stade, puisqu'on a pas de fichier C inclus dans notre projet contenant une fonction `main()` ...

AJOUTER NOTRE FICHIER CONTENANT une fonction main()

créer un nouveau
fichier

taper le code ci contre

sauvegarder en `gpio.c`

la coloration syntaxique
doit apparaître :

Si elle n'apparaît pas,
ce n'est pas un fichier `.c`
et vous ne le verrez pas
à l'étape suivante !
il faut alors aller
changer l'extension, ou
sauvegarder à nouveau.

```
1 unsigned char test=0;
2
3 void init_proc(void)
4 {
5 }
6 int main (void)
7 {
8   init_proc();
9   while(1)
10   { test++;
11   }
12 }
```

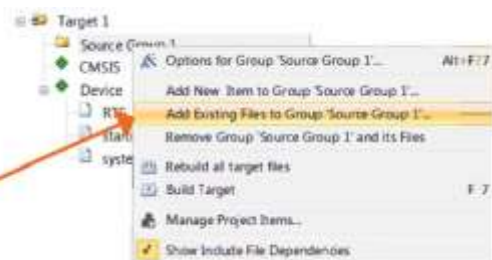
```
unsigned char test=0;

void init_proc(void)
{
}

int main (void)
{
  init_proc();
  while(1)
  { test++;
  }
}
```

AJOUTER LE FICHIER AU PROJET

clic droit sur Source Group 1



Avant

si on compile, on a toujours l'erreur du main manquant

Après ajout du fichier gpio.c contenant la fonction main ()

si on compile, on n'a plus l'erreur du main manquant

on peut en avoir d'autres si on a mal recopié ... regarder les mots soulignés en rouge dans le code (explications page suivante sur les erreurs)



A partir d'ici vous avez un projet fonctionnel pour développer du code en langage C pour le MCU STM32F103RB

2 Découverte de la chaîne de compilation

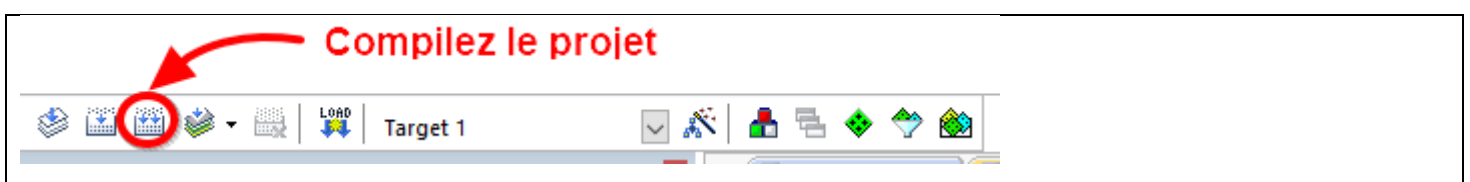
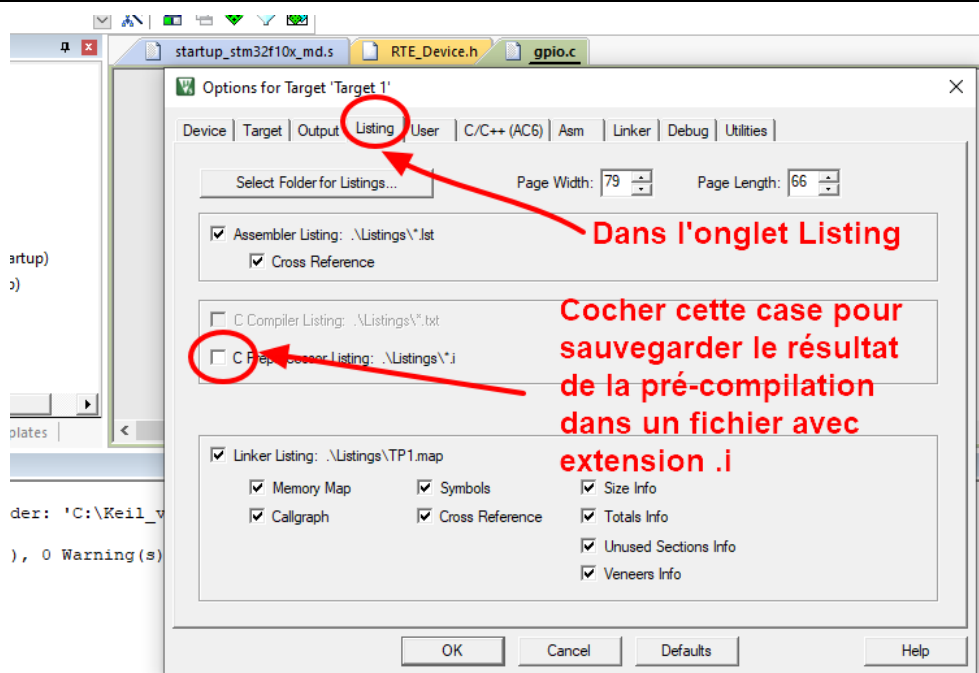
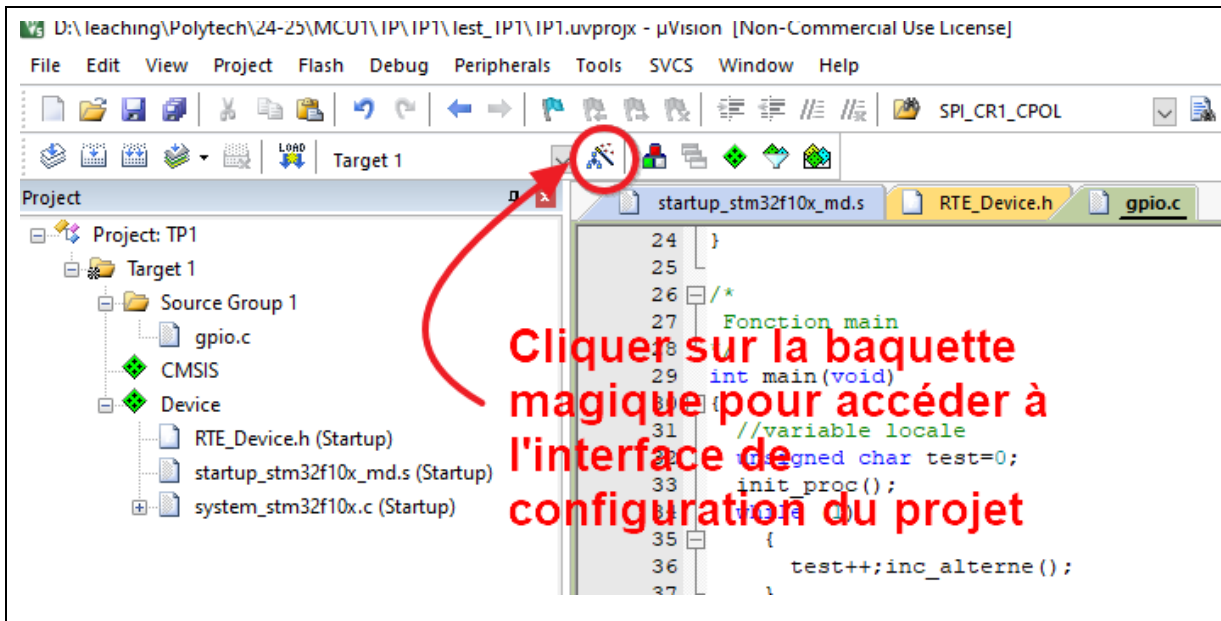
2.1 La précompilation

Modifier le code du fichier **gpio.c** avec le code ci-dessous

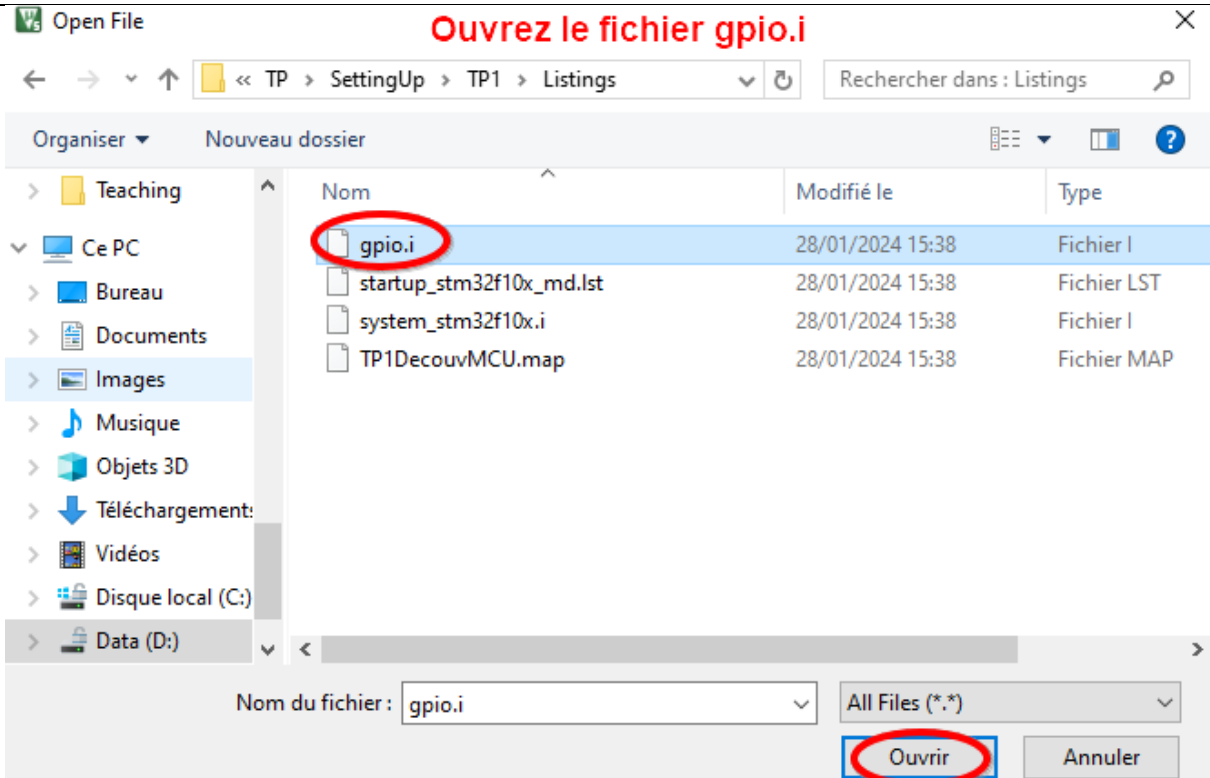
```
//variables globales
unsigned char mavar8bits =0;
short mavar16bits = 0;
int mavar32bits = 0;
#define CAS_8 0
#define CAS_16 1
#define CAS_32 2

void init_proc(void)
{
}
void inc_alterne(void)
{
//variable statique
static unsigned char mae=0;
    switch (mae)
    {
        case CAS_8: mavar8bits++; break;
        case CAS_16: mavar16bits++; break;
        case CAS_32: mavar32bits++; break;
    }
    mae++;
    if (mae ==3) {mae=0;}
}

/*
Fonction main
*/
int main(void)
{
    //variable locale
    unsigned char test=0;
    init_proc();
    while (1)
    {
        test++;inc_alterne();
    }
}
```



Ouvrez le fichier gpio.i



```
//variables globales
unsigned char mavar8bits =0;
short mavar16bits = 0;
int mavar32bits =0;
#define CAS_8 0
#define CAS_16 1
#define CAS_32 2
void init_proc(void)
{
}
void inc_alterne(void)
{
//variable statique
static unsigned char mae=0;
switch (mae)
{
case CAS_8: mavar8bits++; break;
case CAS_16: mavar16bits++; break;
case CAS_32: mavar32bits++; break;
}
mae++; if (mae ==3) {mae=0;}
}
int main(void)
{
//variable locale
unsigned char test=0;

init_proc();
while(1)
{
test++;inc_alterne();
}
}
```

CODE C

Que deviennent ?

-- Les commentaires

-- Les #define

-- Le code C

A TESTER :

rajouter une macro

#define ADD(a,b) a+b

et le code

test = ADD(test, test);

```
unsigned char mavar8bits =0;
short mavar16bits = 0;
int mavar32bits =0;

void init_proc(void)
{
}
void inc_alterne(void)
{
static unsigned char mae=0;
switch (mae)
{
case 0: mavar8bits++; break;
case 1: mavar16bits++; break;
case 2: mavar32bits++; break;
}
mae++; if (mae ==3) {mae=0;}
}
int main(void)
{
unsigned char test=0;

init_proc();
while(1)
{
test++;inc_alterne();
}
}
```

fichier .i

2.2 La cartographie mémoire – fichier MAP

Ouvrir dans le répertoire listing le fichier .map du projet

rechercher les éléments ci dessous , certains sont trouvables, d'autres pas

Symbol Name	Value	Op	Type	Size	Object(Section)
-------------	-------	----	------	------	-----------------

RESET

Reset_Handler

HEAP

STACK

main

init_proc

inc_alterne

ADD

mavar8bits

mavar16bits

mavar32bits

mae

test

DEFINIR COMMENT SONT GERES EN MEMOIRE LES DIVERSES ETIQUETTES

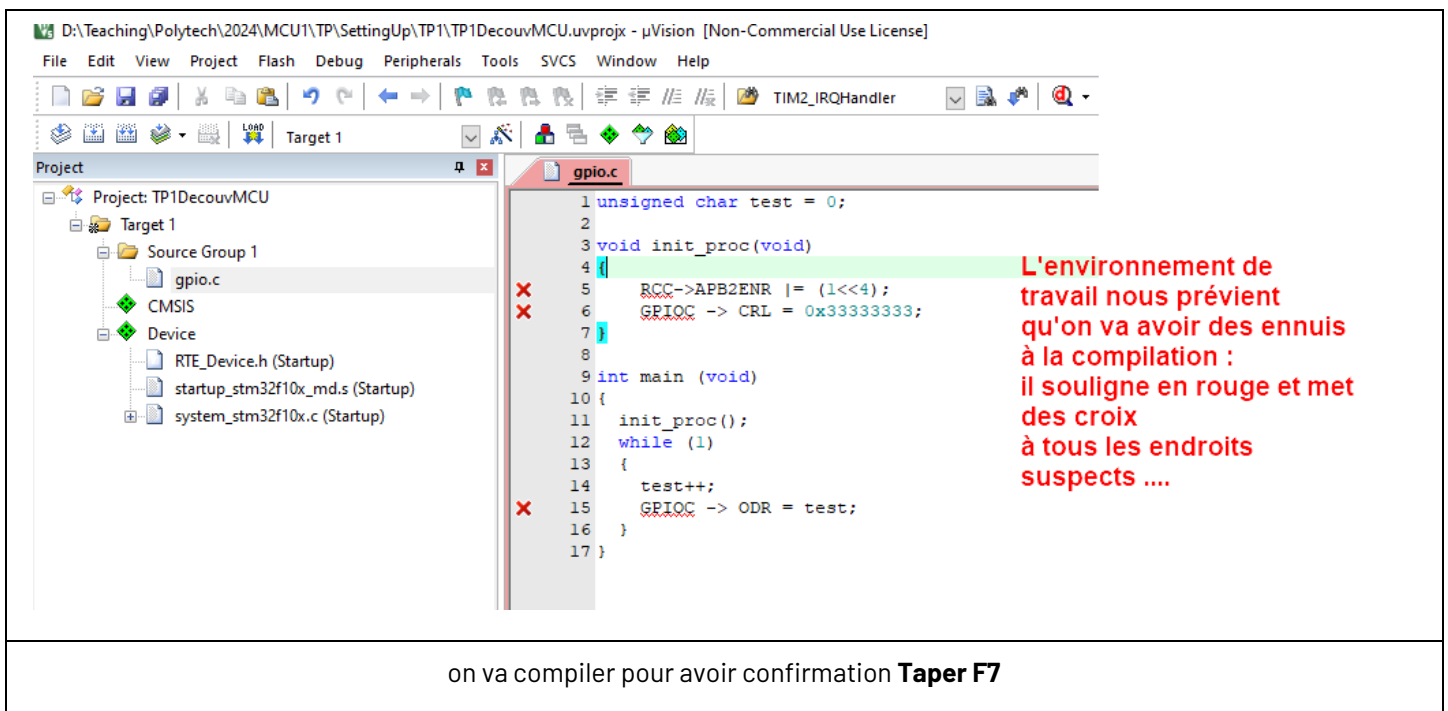
3 Premiers pas avec le STM32F103RB

Modifier le code précédent pour initialiser les 8 bits de poids faible du PORTC en SORTIE avec la technologie PUSH-PULL, et dans la fonction main faire compter le PORT.

```
unsigned char test = 0;

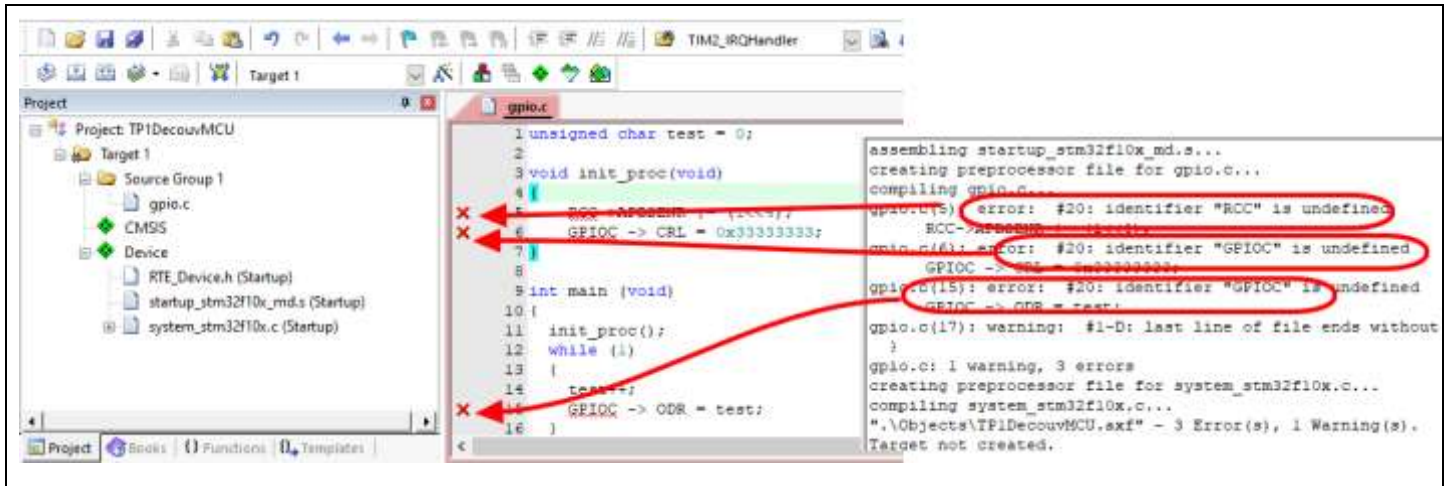
void init_proc(void)
{
    RCC->APB2ENR |= (1<<4);
    GPIOC -> CRL = 0x33333333;
}

int main (void)
{
    init_proc();
    while (1)
    {
        test++;
        GPIOC -> ODR = test;
    }
}
```



L'environnement de travail nous prévient qu'on va avoir des ennuis à la compilation : il souligne en rouge et met des croix à tous les endroits suspects

on va compiler pour avoir confirmation **Taper F7**

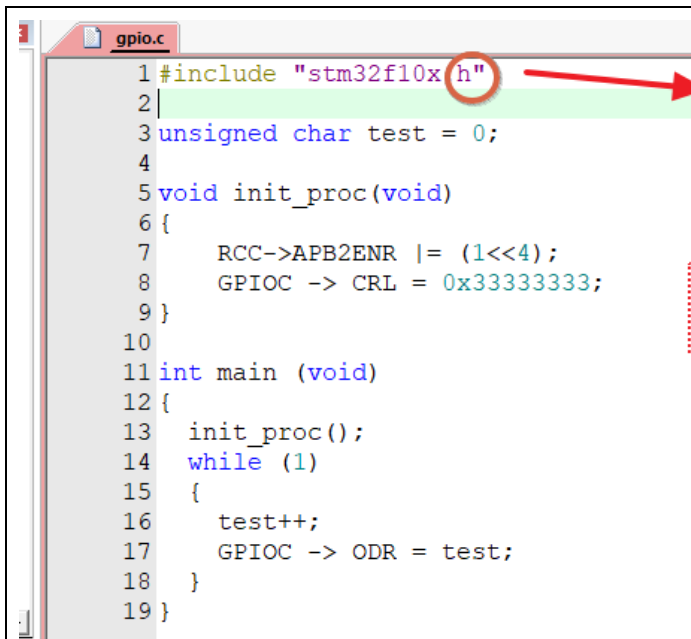


En cliquant sur les erreurs, on est amené directement sur les lignes qui posent problème



LE COMPILATEUR NE CONNAIT PAS LES REGISTRES ??????

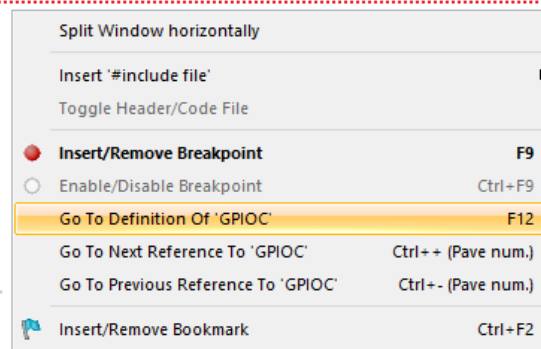
normal : il faut inclure un fichier .h qui les décrit ...



à l'instant où l'on pose ce " final, tous les signalements d'erreur

On va tenter de regarder maintenant le rôle de ce fichier include:

faire un clic droit (clic contextuel) sur GPIOC et : choisir Go To Definition Of 'GPIOC'



La compilation a lieu maintenant sans erreurs et sans warning

/** General Purpose Input/Output (GPIO) register structure definition */

en cherchant la déclaration de **GPIOC**, on tombe sur une cascade de **#define** et la définition de la structure ci-dessous

```

1281 #define SRAM_BASE          ((uint32_t)0x20000000) /*!<
1282 #define PERIPH_BASE        ((uint32_t)0x40000000) /*!<
1283
1284 #define SRAM_BB_BASE       ((uint32_t)0x22000000) /*!<
1285 #define PERIPH_BB_BASE     ((uint32_t)0x42000000) /*!<
1286
1287 #define FSMC_R_BASE        ((uint32_t)0xA0000000) /*!<
1288
1289 /*!< Peripheral memory map */
1290 #define APB1PERIPH_BASE    PERIPH_BASE
1291 #define APB2PERIPH_BASE    (PERIPH_BASE + 0x10000)
1292 #define AHBPERIPH_BASE     (PERIPH_BASE + 0x20000)
1293
1321 #define AFIO_BASE          (APB2PERIPH_BASE + 0x0000)
1322 #define EXTI_BASE          (APB2PERIPH_BASE + 0x0400)
1323 #define GPIOA_BASE         (APB2PERIPH_BASE + 0x0800)
1324 #define GPIOB_BASE         (APB2PERIPH_BASE + 0x0C00)
1325 #define GPIOC_BASE         (APB2PERIPH_BASE + 0x1000)
1326 #define GPIOD_BASE         (APB2PERIPH_BASE + 0x1400)
1327 #define GPIOE_BASE         (APB2PERIPH_BASE + 0x1800)
1328 #define GPIOF_BASE         (APB2PERIPH_BASE + 0x1C00)
1329 #define GPIOG_BASE         (APB2PERIPH_BASE + 0x2000)
1330
1416 #define GPIOA              ((GPIO_TypeDef *) GPIOA_BASE)
1417 #define GPIOB              ((GPIO_TypeDef *) GPIOB_BASE)
1418 #define GPIOC              ((GPIO_TypeDef *) GPIOC_BASE)
1419 #define GPIOD              ((GPIO_TypeDef *) GPIOD_BASE)
1420 #define GPIOE              ((GPIO_TypeDef *) GPIOE_BASE)
1421 #define GPIOF              ((GPIO_TypeDef *) GPIOF_BASE)
1422 #define GPIOG              ((GPIO_TypeDef *) GPIOG_BASE)

```

on voit qu'écrire dans un registre :

GPIOC->CRL = 0x33333333;

revient à modifier le contenu d'une case mémoire dont l'adresse est calculée par des sommes successives :

adresse de base (0x40000000 + 0x00010000 + 0x00001000 + 0x00000000)

pour avoir l'adresse finale : 0x40011000

c'est dans cette case mémoire qu'on viendra mettre la valeur

On retrouve cette adresse dans la doc du STM32F103

SANS LE FICHIER "stm32f10x.h" le compilateur n'a aucune information sur ce qu'on souhaite.

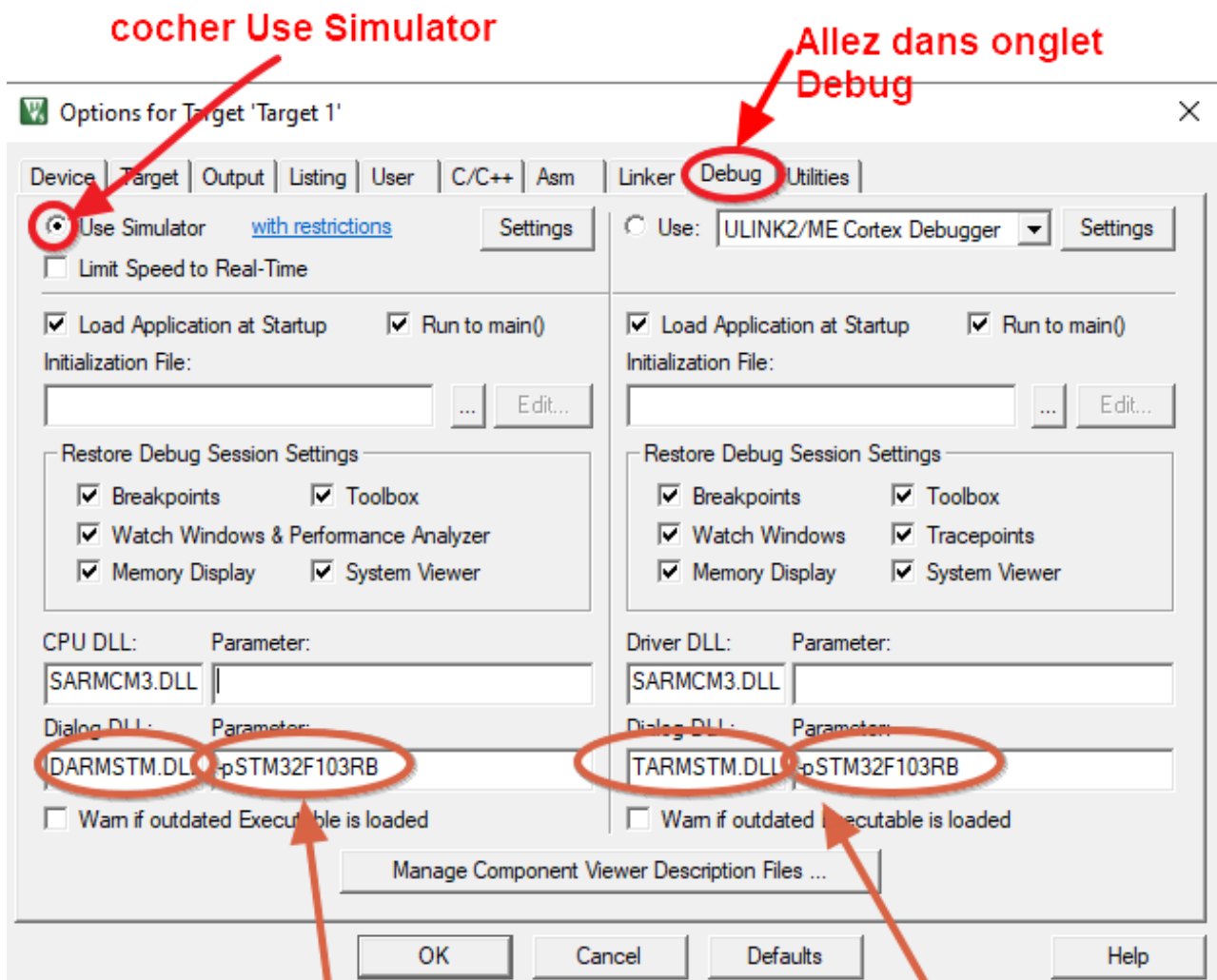
```

1009 typedef struct
1010 {
1011     __IO uint32_t CRL; 0x00
1012     __IO uint32_t CRH; 0x04
1013     __IO uint32_t IDR; 0x08
1014     __IO uint32_t ODR; 0x0C
1015     __IO uint32_t BSRR; 0x10
1016     __IO uint32_t BRR; 0x14
1017     __IO uint32_t LCKR; 0x18
1018 } GPIO_TypeDef;

```

3.1 Simulation de fonctionnement du STM32F103RB

Pour simuler notre projet, il faut retourner dans options for Target, onglet Debug (ALT F7) debug



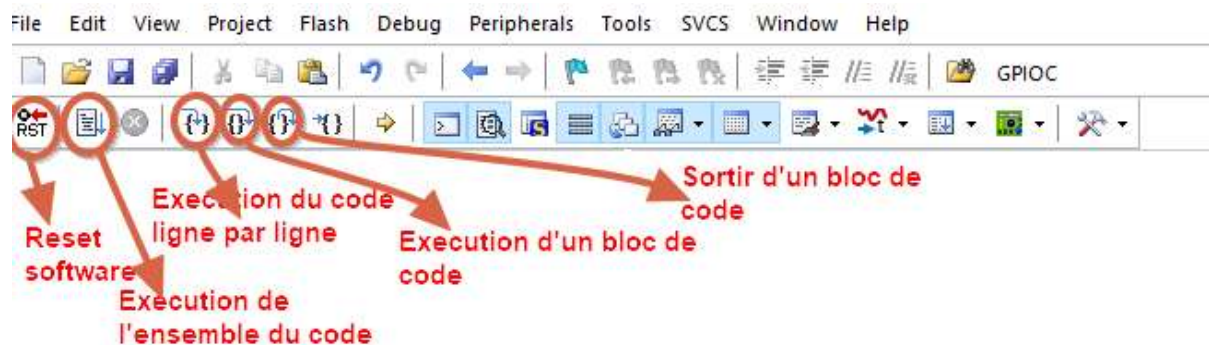
Si les champs sont erronés , corriger à la main

3.1.1 Simulation en mode pas à pas et vérification du contenu des registres de configuration des Pins

Cliquez sur le bouton Debug permet pour basculer dans le mode DEBUG

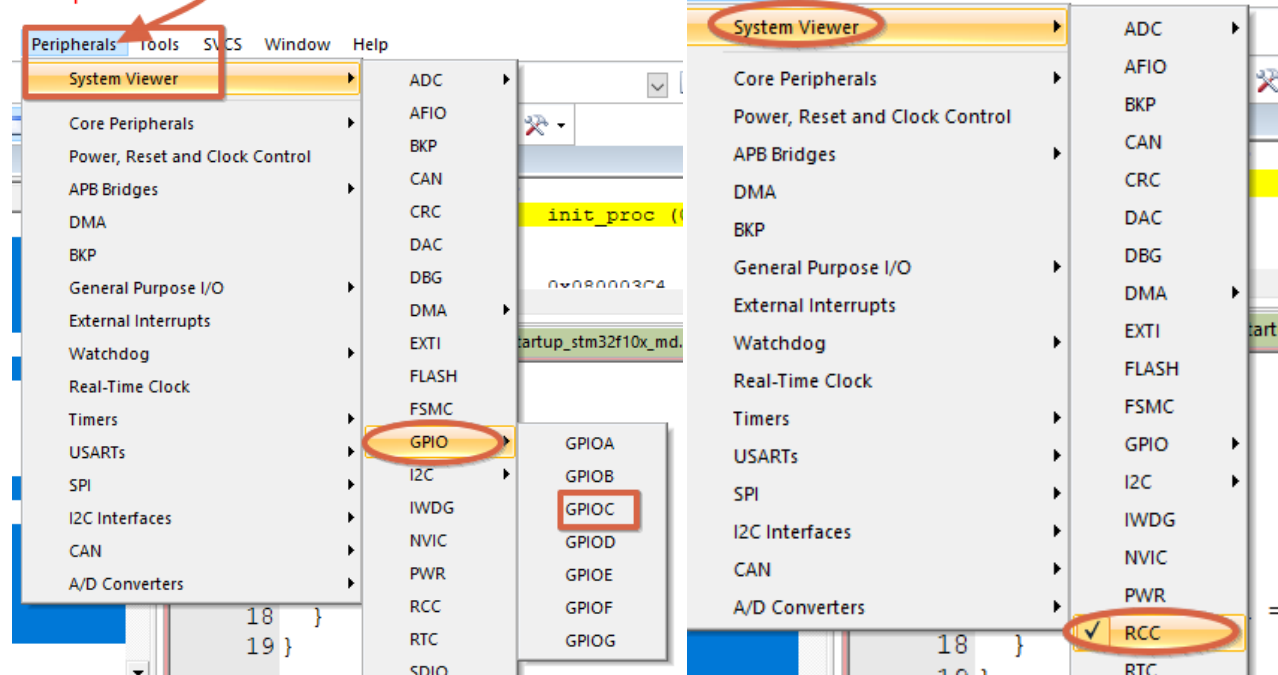


Les boutons disponibles en mode DEBUG.



Afficher les registres du GPIOC et du périphérique RCC. Avec le mode PAS à PAS vérifier que les valeurs inscrites sont bien celles prévues par le code. Vérifier que les valeurs du registre de sortie GPIOC->ODR change conformément au code.

Allez dans l'onglet Peripherals



On obtient les vues ci-dessous

The screenshot shows an IDE with three main panes. The top pane is the 'Disassembly' view, showing assembly instructions for the 'init_proc' function. The middle pane is the 'C' code view, showing the implementation of 'init_proc' in 'gpio.c'. The bottom pane is the 'GPIOC' peripheral view, showing the values of various registers.

Disassembly View:

```

13:      init_proc();
14:      while (1)
15:      {
16:          test++;
17:          GPIOC -> ODR = test;
18:      }
19:  }
  
```

C Code View (gpio.c):

```

10
11 int main (void)
12 {
13     init_proc();
14     while (1)
15     {
16         test++;
17         GPIOC -> ODR = test;
18     }
19 }
  
```

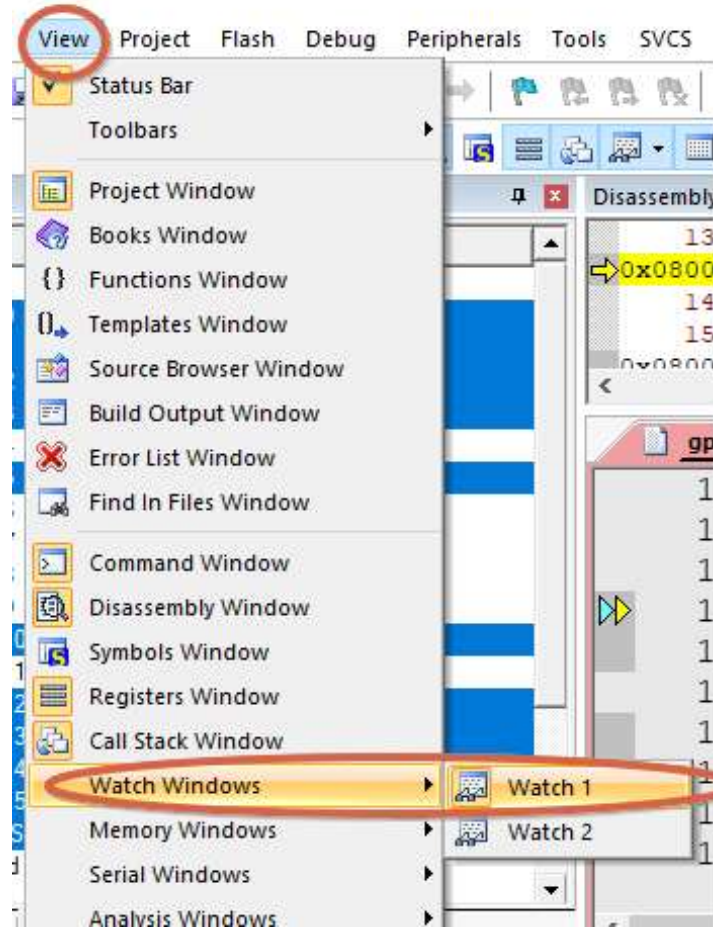
GPIOC View:

Property	Value
CRL	0x44444444
CRH	0x44444444
IDR	0
ODR	0
BSRR	0
BRR	0
LCKR	0

At the bottom of the IDE, the 'RCC' and 'GPIOC' tabs are visible and circled in red.

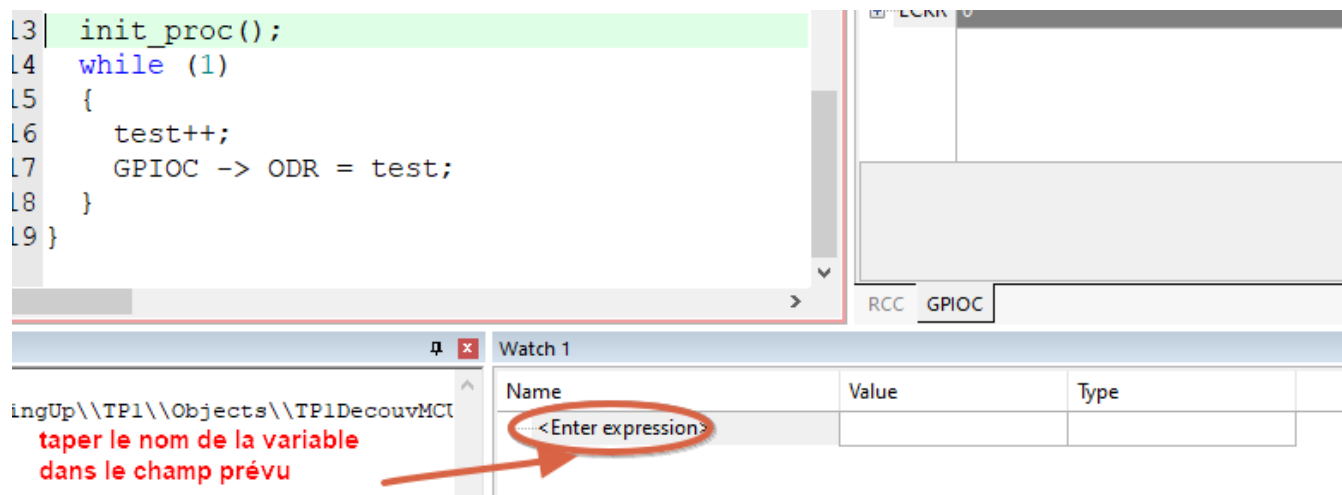
Afficher la variable globale test et vérifier que sa valeur est égale à celle du registre GPIOC->ODR.

1 Allez dans l'onglet View



2

Choisir Watch 1 dans le menu Watch Window



Afficher les chronogrammes des pins du PORTC avec l'outil « Logic Analyzer »

1 Sélectionner Logic Analyzer en cliquant sur le bouton "System Analyzer"

2 Cliquez SETUP

3 Taper PORTC.0 pour afficher le chronogramme du bit B0 du GPIOC->ODR

4 Terminer la configuration avec les valeurs des paramètres

Après exécution du code vous obtenez une fenêtre avec le chronogramme correspondant

